

TrainAudit: Silent Error Detection for LLM Training via Verified Guarded Relational Constraints

Qingsong Cai*
Sichuan University
Chengdu, China
qingsong1010@gmail.com

Shikai Li*
West China Hospital, Sichuan
University
Chengdu, China
lishikai@wchscu.edu.cn

Zhengmao Ye
Sichuan University
Chengdu, China
yeyzhengmao@gmail.com

Mingjie Tang
Sichuan University
Chengdu, China
tangrock@gmail.com

Ran Tao
iQuest Lab
Beijing, China
rtao02@iquestlab.com

Bryan Dai
iQuest Lab
Beijing, China
cbdai@iquestlab.com

ABSTRACT

We introduce TRAINAUDIT, a production-oriented system designed to efficiently and reliably detect silent errors during large language model (LLM) training. Such errors violate prescribed training behavior without crashing the program, often leaving per-step metrics apparently normal while degrading model quality and wasting tens of thousands of GPU-hours. Standard monitors provide no online oracle for topology- and phase-dependent relations in runtime state. TRAINAUDIT addresses this gap with *guarded integrity constraints* over training traces, establishing explicit correctness obligations for when and where runtime relations must hold. It further adopts a two-phase auditing architecture that separates offline constraint discovery from deterministic online checking. Offline, candidate constraints are recovered from framework source and confirmed bug fixes and admitted only after verification; online, SQL queries check the live trace, and each non-empty result provides a concrete violation witness. On the frozen *Real-SE* benchmark, TRAINAUDIT detects 17/18 (94.4%) buggy instances and raises no false positives on all 17 corresponding fixed revisions. In a planned seven-day, 256×H800 production run, it surfaced a previously unreported optimizer-routing fault before the first optimizer step; the job had a nominal allocation of approximately 43,000 H800 GPU-hours. Upstream maintainers subsequently confirmed and fixed the fault.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Software and its engineering** → **Software reliability**; *Software testing and debugging*.

KEYWORDS

distributed training, large language models, silent errors, data quality, integrity constraints, constraint mining

PVLDB Reference Format:

Qingsong Cai, Shikai Li, Zhengmao Ye, Mingjie Tang, Ran Tao, and Bryan Dai. TrainAudit: Silent Error Detection for LLM Training. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

*Qingsong Cai and Shikai Li contributed equally to this work.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/vivia-an/trainaudit-vldb-artifact>.

1 INTRODUCTION

Training a large AI model is a long-running distributed state transition. At each step, thousands of GPUs compute partial gradients, exchange tensors through collective communication [16], update optimizer state, restore or write checkpoints, and advance framework counters under a specific parallel topology. At frontier scale, a single run occupies thousands of GPUs for weeks [14]; an undetected silent error corrupts the trajectory for the remainder, wasting tens of thousands of GPU-hours before the damage—if ever—surfaces.

A *silent error*¹ occurs when this transition deviates from the intended training algorithm without becoming obviously invalid. Such errors include a rank skipping synchronization, an accumulated gradient being overwritten, only part of a checkpoint being restored, or an optimizer counter advancing in the wrong phase. They are *semantic* in nature—they violate what the training algorithm prescribes, not what the runtime enforces—yet *silent* in manifestation, because existing runtime infrastructure checks only structural validity (types, shapes, exceptions) and has no interface for algorithmic correctness.

Figure 1 shows a concrete example from DeepSpeed ZeRO-2 [25, 26]. An off-by-one error in the `micro_step_id` counter resets a gradient buffer when it should be accumulated. The run continues, the tensors have legal shapes and dtypes, and the loss curve remains nearly unchanged. Yet the live execution already contains evidence of the error: the current micro-step, the buffer operation, the accumulation phase, and the optimizer boundary. None of these fields is a reliable alarm by itself; together, however, they define a relation that correct training must satisfy: in a given accumulation phase, the buffer operation must match the intended update rule.

emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

¹Following prior work [12, 13], we use *silent errors* to denote software defects that silently corrupt training state without raising exceptions—distinct from hardware-induced silent data corruption (SDC) [6, 10], which is orthogonal and addressed by ECC.

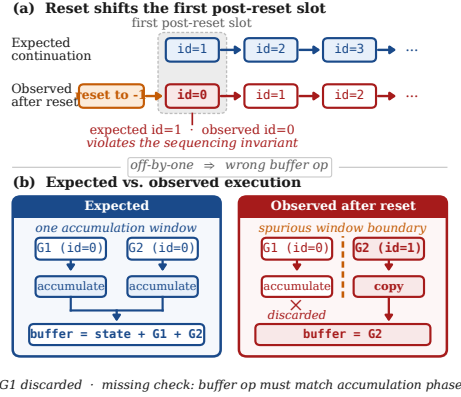


Figure 1: DeepSpeed ZeRO-2 silent error: an off-by-one micro_step_id overwrites rather than accumulates a gradient buffer.

Detecting such errors therefore requires recovering the semantic correctness conditions that current monitoring leaves unchecked.

Existing detectors observe this transition through limited oracle sources. Metric-based monitors are easy to deploy, but they watch downstream symptoms such as loss spikes, NaNs/Infs, gradient-norm outliers, and throughput anomalies. Healthy-trace approaches can learn useful execution regularities [13], but regularity in previous traces is not the same as a framework-level correctness obligation under a new topology or phase. Reference-based checking obtains a stronger oracle by comparing intermediate tensors against a trusted or alignable implementation [12], but such a reference is often unavailable for production-scale runs. Plan verification can reason about whether a distributed execution plan matches a logical specification [18], but it does not audit the dynamic state transitions that occur as counters, checkpoint state, optimizer metadata, and communication groups evolve during training. Our insight is that the hard silent errors are not signal-free; their signal is stranded below current monitoring interfaces.

The gap between the runtime evidence already present and the interfaces exposed by existing detectors creates a two-sided data-management problem. On one side, scalar monitoring observes too little: loss, gradient norm, NaN/Inf counters, and throughput relations on which the signal depends. On the other, recording everything does not by itself provide a correctness oracle. Raw tracing can materialize millions of records about tensor states, optimizer counters, communication events, checkpoint state, rank groups, topology lineage, and framework metadata, yet it does not say which combinations are illegal. What is needed, then, is a way to locate the meaningful signal within this rich runtime state—observing more than scalar metrics, yet with enough semantic guidance to avoid being lost in millions of undifferentiated records.

The central obstacle is that these runtime records do not come with an explicit specification of correct training behavior. We observe, however, that framework source code and confirmed bug fixes jointly encode both what can go wrong and what must hold:

each fix implicitly reveals a correctness condition that was previously unwritten. This paper starts from these latent obligations and asks whether they can be recovered systematically and operationalized as online checks, without requiring a trusted reference run. Two difficulties stand between this idea and a deployable detector: each recovered obligation is valid only under specific topology and phase conditions—enforced unconditionally, it floods the operator with false alarms on correct runs—and recovering obligations at scale from framework artifacts requires automation that is itself error-prone.

We propose TRAINAUDIT, which addresses both by separating obligation *discovery* (offline, LLM-assisted, verified against counterexamples before deployment) from obligation *checking* (online, fully deterministic SQL queries over a relational training trace), so that the online path contains no LLM and no unverified rule. This separation is what makes model-proposed semantics safe to deploy: a model may contribute a candidate obligation, but a deterministic query over runtime evidence decides whether an execution violates it.

Across a 392-record silent-error corpus, a frozen catalog snapshot matches **92.3%** of the locked held-out bugs, with all three predicates groundable for **91.0%**. On paired reproductions of real buggy and fixed revisions, TRAINAUDIT detects **17/18 (94.4%)** buggy cases and raises no false positives on all 17 corresponding fixed revisions.

In a planned seven-day, 256×H800 production job, a verified rule identified an optimizer-routing fault before the first optimizer step (approximately 43,000 nominal H800 GPU-hours); maintainers subsequently confirmed and fixed the fault [33].

In summary, we make the following contributions:

- 1) We define a *training trace data model*, a relational data model for distributed AI training executions covering tensor state, optimizer state, call events, communication/checkpoint events, rank groups, and topology lineage.
- 2) We formalize silent-error detection as checking *runtime-guarded integrity constraints* over live training traces, where each constraint specifies which runtime fields to read, which tuples can be compared under the current topology, when the relation is valid, and what violation predicate to execute.
- 3) We develop a reproducible template-induction protocol and a *verified mining and execution* pipeline: under a matched free-form control, a versioned relation catalog raises validated-rule yield while evolving with new framework evidence; counterexample checks, healthy-run replay, and topology-aware validation govern acceptance before deterministic SQL execution.
- 4) We build *Real-SE*, a reproducible benchmark of paired buggy/fixed revisions and an observability-boundary case, to evaluate detection and fixed-side precision (§5.2); full protocols and per-case outcomes appear in the supplementary material [1].

2 BACKGROUND AND MOTIVATION

The prevailing way to diagnose a silent error is still manual: for each incident, an engineer inspects the runtime state, relates it to the framework source and training algorithm, and reconstructs what the execution should have done. This process is costly and ad hoc, and the resulting diagnosis is rarely reusable across runs. Yet its success reveals an important opportunity: silent errors often leave

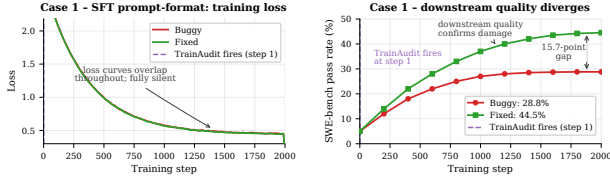


Figure 2: Production silent error (Appendix H, Case 1): losses overlap for 2,000 steps while downstream quality diverges 15.7 points; the rule fires at step 1.

a *witness* in the runtime state; what is missing is a reusable correctness *obligation* that gives this evidence meaning. Such obligations must be recovered from training code, algorithmic semantics, or confirmed fixes, then converted into checks that can be applied automatically. The remainder of this section develops this observation: §2.1 defines the target errors and explains why existing oracle sources fail to exploit such witnesses reliably, while §2.2 identifies the three data-management challenges (C1–C3) in turning latent obligations into practical online checks.

2.1 Silent Errors and Why Existing Approaches Miss Them

A *silent error* is a fault in distributed training that produces numerically valid tensors and raises no exception, yet drives the run off the trajectory that the training algorithm prescribes. Root causes include software bugs (e.g., a missing all-reduce), framework misuse, and configuration drift. Their effects can first become observable in communication, gradient handling, optimizer updates, checkpointing, or training-loop control, across one or more DP/PP/TP/EP dimensions of hybrid parallelism (Appendix I gives the phase and topology panorama).

Despite this breadth, outcome metrics may surface the damage only after it has propagated: loss and gradient norm can track a healthy baseline long after the training state has been corrupted (Figure 2). TRAINAUDIT therefore targets errors that leave a sound, observable witness in the instrumented runtime state even while per-step metrics remain plausible. Faults whose only manifestation is asymptotic convergence drift fall outside this scope; §3.3 formalizes why no guarded predicate over the current schema can soundly separate them from correct executions.

A silent-error detector requires a *detection oracle*: a source of truth against which observed runtime state is judged correct or corrupt. Each choice of oracle imposes a structural ceiling: outcome-metric monitors fire only after the macro curve bends, because aggregation can erase a fault’s fine-grained signature; behavioral trace templates infer invariants—including topology-conditioned ones—from example runs, so they cover only what those runs exercise; differential testing needs a trusted, alignable reference and runs offline; and plan verification covers only what is decidable before the first step. In short, runtime evidence alone is not an oracle: existing approaches either discard its fine-grained structure, generalize only from previously observed executions, require a trusted reference, or cannot validate the execution online. Table 1 summarizes these oracle choices and their structural limitations;

Table 1: Silent-error detectors differ in the source and availability of their correctness oracle.

Method	Mode	Oracle	Key limitation
Loss / NaN monitor	Online	Outcome metrics	Late, uninformative
TrainCheck	Online	Example-run invariants	Bounded by example runs
TTrace	Offline	Reference impl. diff	Needs alignable reference
TrainVerify	Static	Logical specification	Plan-only, no runtime
TRAINAUDIT	Online	Bug-derived constraints	Requires source-grounded constraint recovery

§5.2 evaluates their observable consequences under a case-aligned protocol.

TRAINAUDIT occupies the unexplored cell of this design space: it derives deployable checks from *historical bug fixes* rather than from example executions, so a check’s authority comes from source-level evidence and fix replay rather than from the runs available for invariant inference, while remaining online and requiring no trusted reference. This position is feasible because of a structural property established in §3—silent errors repeatedly violate reusable relations over runtime training facts, even as the concrete template catalog grows with framework code. Turning those relations into trustworthy online checks requires overcoming three obstacles (§2.2).

2.2 Challenges

C1: Correctness obligations require precise runtime guards. Whether a relation over training state must hold is itself a conditional integrity constraint: it holds on some tuples and not others. Within a single tensor-parallel run, `layernorm.weight` is replicated across ranks and must be bitwise equal, whereas `query_key_value.weight` is partitioned and must differ—yet a single cross-rank equality check matches both. Enforced unconditionally, it flags every legitimately partitioned tensor, so a constraint carrying no guard for tensors, topology, and phase is unusable in production (§3.2).

C2: Correctness obligations are latent and must be recovered safely. The constraints that define correct training are never written down as such: they are latent in the framework’s implementation and the algorithm’s mathematics, and only their *violations* ever surface—as a fixed bug or a documented caveat. This rules out profiling them from a data instance the way integrity constraints usually are: a single clean run exhibits countless regularities, but a regularity that merely happened to hold is not one that must hold, and enforcing it fires false alarms on the next run. The authoritative source is the code and algorithm that generate the data, not the data itself—yet reading a constraint off that source is error-prone, because what the documentation states and what the implementation enforces routinely diverge. A candidate that looks correct on paper is quietly violated by the framework’s own initialization path, so without validation against the actual code a mined library is itself a source of false positives (§4.3).

C3: Verified obligations must be checked at training speed. The check runs online, on the critical path of every training step: a detector that slows the step, or falls behind it, will be turned off. Yet the state worth checking is large—parameters, gradients, and optimizer state across every rank—and inspecting it in full at every step is too expensive to sustain in production (§4.4, §4.5).

These challenges structure the paper. For C1, §3.2 decomposes every deployable constraint into three predicates—measurement schema, agreement topology, validity precondition—whose guards the Invariant Miner instantiates per framework (§4.3). For C2, the Miner treats LLM-proposed candidates as untrusted, admitting a constraint only after counterexample verification, healthy-run replay, and topology-aware scope validation (§4.3). For C3, the Data Collector captures only fields that accepted constraints read (§4.4) and the Verifier prunes rules inapplicable under the active topology (§4.5); overhead is quantified in §5.5. §3 first establishes the empirical foundation: real silent errors reuse guarded relations over runtime training facts, while the catalog of concrete templates evolves with the frameworks.

3 EMPIRICAL STUDY AND PROBLEM FORMULATION

The preceding section identifies the need for a source-grounded online oracle. This section first establishes the empirical premises that make such an oracle possible and then defines the checking problem precisely. §3.1 studies the 392-record upstream evidence corpus to determine where the correctness standard comes from, whether silent errors recur at the level of runtime relations, and when those relations require topology and phase guards. §3.2 translates these findings into guarded relational constraints with three predicates: a measurement schema, an agreement topology, and a validity precondition. §3.3 establishes formal properties of this decomposition, including the schema-relative observability ceiling measured in §5.

3.1 Empirical Study: The Structure of Silent Errors

Corpus construction. We test three premises needed by a source-grounded runtime oracle: the intended relation comes from outside the runtime data, relation types recur across incidents, and their applicability depends on topology and phase. Our evidence is a curated corpus of **392 upstream silent-error records** from Megatron-LM, DeepSpeed, OLMo, and OLMo-core. The released corpus combines a commit-backed collection with an earlier issue/PR-curated collection, deduplicated by upstream incident (Megatron-LM 110, DeepSpeed 123, OLMo 77, OLMo-core 82). Commit diffs and messages are the preferred evidence because they identify the changed training path and a concrete before/after semantics. Issue text is supporting evidence only, since reports often lack the buggy revision, trigger configuration, or an executable oracle.

A candidate is retained when the available evidence supports a semantic correctness failure in a training-critical path rather than documentation, performance, or refactoring, and describes silent continuation as the primary behavior rather than an immediate exception. We require enough information to extract the affected

Table 2: Distribution of the 392-record evidence corpus by primary subsystem, typical parallelism dimension, and first observable loop stage.

Primary label	Records	Parallelism	Loop Stage
Numerical	75	any	Fwd/Bwd
Control Flow	50	any	All
Grad Sync	46	DP	Backward
Checkpoint	32	DP/TP/PP	Save/Load
MoE	28	EP	Fwd/Bwd
Optim State	28	DP/FSDP	Optim Step
Data Loading	28	DP	Pre-step
Communication	26	DP/TP/PP/EP	All
Dtype	24	any	Fwd/Bwd
Loss Comp.	23	any	Forward
Sharding	13	TP/PP	Init/Fwd
LR Schedule	13	any	Optim Step
Offload	6	DP/FSDP	Optim Step
Total	392		

semantic objects and intended relation, but do not treat this screening decision as physical reproduction: only the runnable subsets verify buggy and fixed behavior directly. We assign each record one coarse primary subsystem label by inductive coding and report the distribution in Table 2. These labels are descriptive bins for corpus reporting, not the relation-template space used by TRAINAUDIT; in the table, “any” denotes topology-independent applicability. Appendix B reports provenance strength separately: 289 records carry a fixed-commit identifier, 96 are issue/PR-only records, and 7 retain only a curated source summary. The corpus is therefore heterogeneous evidence for coding and catalog induction, not 392 equally reproduced bugs nor an exhaustive all-commit census.

Finding 1: Recognition requires an external correctness relation. The retained evidence does not identify a fault from an unlabelled runtime record alone; it first supplies an intended relation from the training code, algorithm, or confirmed fix. Runtime state may then contain a witness—for example unequal replicas or an incorrect call count—but it does not encode why those values should agree or how many calls are correct. In the physically replayed subset, ordinary operator-facing signals such as loss and gradient norms can remain plausible despite the violation. We do not claim a bitwise-identical healthy/buggy trace for every record; the corpus supports only this structural claim.

Finding 2: Silent errors recur at the level of runtime relations. Across the coded records, faults from different frameworks repeatedly instantiate the same kinds of obligations—replicated state should agree, scaled quantities should carry the intended factor, producers should precede consumers, and saved state should be restored completely. Yet these relations apply to different semantic objects and under different topology and phase conditions. This motivates an intermediate representation between the coarse corpus labels and framework-specific checks: a catalog of reusable relation templates whose reproducibility and coverage we evaluate in §5.4.

Finding 3: Whether a constraint must hold depends on topology and phase. The two rightmost columns of Table 2 summarize the guard

structure visible even at this coarse coding level. Several labels are tied to specific parallelism dimensions—Grad Sync to DP, Sharding to TP/PP, and MoE to EP—and failures first become observable at different points of the training loop. A deployable check must therefore carry with it the topology under which its relation is required and the phase at which it is valid. §3.2 formalizes exactly this requirement as a three-predicate decomposition.

3.2 Problem Formulation and Three-Predicate Decomposition

Building on §3.1’s finding that the correctness constraint lives outside the data, this subsection formalizes what it means to check such a constraint at runtime and reduces every deployable check to three predicates. We model a training run as a sequence of states and cast a correctness constraint as a *semantic invariant* over them. Each state contains the objects that a silent error can corrupt: model parameters on each rank, gradients, optimizer state, training-loop control variables, framework metadata such as `tensor_model_parallel`, and the active parallel topology τ . A silent error occurs when the actual next state differs from the intended next state, while the program raises no exception.

Definition 3.1 (Silent Error). A silent error at step t is a state $\tilde{s}_{t+1} = f_{\text{actual}}(s_t)$ such that $\tilde{s}_{t+1} \neq f_{\text{ideal}}(s_t)$ and no runtime exception is raised.

Since f_{ideal} is unavailable at runtime, TRAINAUDIT instead checks *semantic invariants*: relations that must hold in every correct training state—replicated parameters should agree across the right ranks, counters should advance with the training loop, and framework metadata should be consistent with the active topology. Crucially, each training state is already materialized as relational trace tuples, so such an invariant becomes a *guarded relational constraint* that TRAINAUDIT evaluates as a query over the trace, where a violation surfaces as a non-empty result. The remaining difficulty is therefore to make each constraint precise enough to deploy, which is what the three-predicate decomposition below provides.

Definition 3.2 (Semantic Invariant). A semantic invariant is a predicate $p : \mathcal{S} \rightarrow \{0, 1\}$ such that $\forall s_0, t : p(f_{\text{ideal}}^t(s_0)) = 1$.

Consider a DP=2 run where a missing `allreduce` causes rank 0 and rank 1 to accumulate different gradient updates silently. In state terms, $\theta_{\text{rank0}} \neq \theta_{\text{rank1}}$ while no exception is raised—a silent error by Definition 3.1. The natural invariant is $p(s) = [\theta_{\text{rank0}} = \theta_{\text{rank1}}]$, which is violated (Definition 3.2). But three questions must be answered to make it useful in production: *what exactly to compare?* (parameter checksums, not raw tensors), *on which parameters?* (DP-replicated weights only—TP-sharded `query_key_value` weights are *supposed* to differ), and *when?* (after step-1 synchronization, not at initialization).

Every deployable constraint must answer these three questions, so we decompose each constraint into three predicates:

$$p(s) = \pi_{\text{schema}}(s) \wedge \pi_{\text{topo}}(s; \tau) \wedge \pi_{\text{precond}}(s; t). \quad (1)$$

π_{schema} selects the fields of s to read, such as parameter checksums, gradient norms, or function-call counts. π_{topo} encodes which ranks or objects should agree under the active topology τ . π_{precond} guards the check by training step and framework

metadata, preventing valid states such as intentionally sharded tensors from being reported as violations. The three predicates are jointly necessary: dropping any one leaves the others without production usability—a bare cross-rank comparison misfires on correctly-sharded tensors, and an unguarded equality check fires before step-1 synchronization—which our drop-one ablation quantifies (§5.3). This decomposition is the formal answer to C1: the guards are part of the constraint, not side information attached after the fact.

3.3 Formal Properties of Guarded Constraints

We briefly formalize the three-predicate decomposition as a Boolean-algebra fragment over the trace schema, establishing three properties that §5 measures directly. Write $G_c = \pi_{\text{topo}} \wedge \pi_{\text{precond}}$ for the *guard* of constraint c . The *violation set* on trace D is

$$V_c(D) = \{ \text{tup} \in D : G_c(\text{tup}) \wedge \neg \pi_{\text{schema}}(\text{tup}) \}, \quad (2)$$

and c fires on D iff $V_c(D) \neq \emptyset$. For relational checks, π_{schema} is evaluated per *comparison group*—the guarded tuples sharing the constraint’s grouping key, e.g., all replicas of one parameter at one step—and $V_c(D)$ contains the tuples of every violating group (single-tuple checks are singletons); this is exactly the WHERE/GROUP BY/HAVING shape the Verifier compiles to (§4.5).

PROPOSITION 3.3 (SOUNDNESS). If p_c is a semantic invariant (Definition 3.2) and c fires on the trace $\tilde{D}_t$ of an actual execution, then $\tilde{s}_t$ lies on no ideal trajectory.

Immediate: firing implies $p_c(\tilde{s}_t) = 0$, which by Definition 3.2 excludes $\tilde{s}_t$ from every ideal trajectory.

PROPOSITION 3.4 (GUARD MONOTONICITY). If c_1, c_2 share the same schema condition and $G_1 \Rightarrow G_2$, then $V_{c_1}(D) \subseteq V_{c_2}(D)$ for every trace D .

Immediate from the subset relation on the guard conjuncts. Dropping any conjunct from a guard can therefore only weakly increase the false-positive count on a fixed clean corpus; §5.3 measures the magnitude of each guard’s effect.

The result below characterizes the schema-relative limit without assuming that every numerical difference is itself a sound, reference-free oracle.

THEOREM 3.5 (SCHEMA-RELATIVE OBSERVABILITY CEILING). Let $\text{proj}_\Sigma(D)$ denote a trace projected onto schema Σ , and let $\mathcal{L}_\Sigma$ be the set of such projections admitted by correct executions under the applicable topology and phase guards. If $\text{proj}_\Sigma(\tilde{D}) \in \mathcal{L}_\Sigma$ for a faulty trace $\tilde{D}$, no sound guarded predicate over Σ can distinguish $\tilde{D}$ from all correct executions.

PROOF. If $\text{proj}_\Sigma(\tilde{D}) \in \mathcal{L}_\Sigma$, some correct execution has the same observable projection. Any predicate over Σ has the same value on the two projections, so a predicate that fires on $\tilde{D}$ also fires on a correct execution and is not sound. The ceiling therefore lies at the schema, not at the predicate language: any separator expressible over Σ can be conjoined with the applicable guards. $\square$

The bound is tight in practice: O-003’s sub-percent z-loss change is present numerically, but remains within the values admitted by clean executions and has no sound reference-free separator under

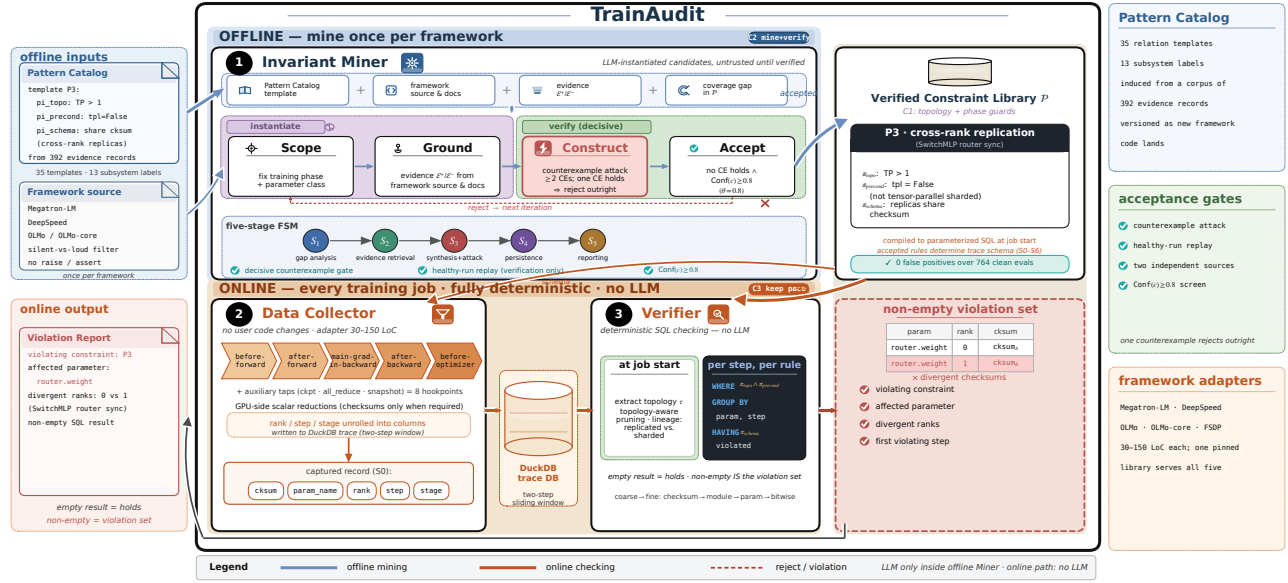


Figure 3: TRAINAUDIT architecture: offline rule discovery and online deterministic trace checking. No LLM runs online.

the current schema. It therefore remains observationally indistinguishable from some correct execution under the available online information (§5.2). This ceiling is a property of the instrumented schema, not of the constraint language: no sound guarded predicate over Σ can close it, so widening Σ —not enriching the rules—is the only remedy.

How TRAINAUDIT mines such three-predicate constraints from framework artifacts (C2) and executes them at training pace (C3) is the subject of §4.

4 TRAINAUDIT: MINING AND CHECKING VERIFIED CONSTRAINTS

TRAINAUDIT follows a four-stage pipeline: ❶ the **Pattern Catalog** provides reusable templates (§4.2); ❷ the **Invariant Miner** grounds and verifies them (§4.3); ❸ the **Data Collector** records the required fields (§4.4); and ❹ the **Verifier** checks the resulting trace (§4.5). Across all four stages, a rule must remain broad enough to capture a recurring silent-error pattern, yet narrow enough to stay silent on correct executions.²

4.1 Architecture and Running Example

TRAINAUDIT evaluates constraints over a relational *training trace*, so that a silent error surfaces as a non-empty query result. Figure 3 separates offline constraint preparation from online enforcement. Facing C1 and C2, the offline Invariant Miner grounds catalog templates with deployable topology and phase guards, then admits them through counterexample verification and healthy-run replay. To address C3, the online Data Collector materializes only the fields accepted rules read, while the Verifier executes compiled SQL under

topology-aware pruning and reports the violating constraint, ranks, and step. Accepted constraints therefore execute deterministically; LLM output never enters the runtime verdict path.

The system attaches through one context-manager wrapper with no user code changes. A lightweight per-framework adapter maps abstract lifecycle points to framework entry points. Across the evaluated frameworks, the complete adapter implementations comprise 30–150 LoC (§5.6).

A cross-rank replication bug in Megatron-LM’s SwitchMLP router weights provides the running example. A missing synchronization leaves two replicas that should be bitwise equal silently divergent. The correctness relation is easy to state yet unsafe to deploy unguarded. Replicated parameters must share a checksum across TP ranks, but intentionally sharded parameters must not be compared at all; the bare check therefore flags 57 correctly sharded `query_key_value.weight` tensors on a clean `TP = 2` run. TRAINAUDIT scopes the cross-rank replication template before deployment—comparing checksums only when `TP > 1` and the parameter is not tensor-parallel sharded—so the same check stays silent on this clean run yet returns the divergent `router.weight` in the buggy run.

4.2 Pattern Catalog: From Corpus Evidence to Reusable Rule Templates

Why a catalog is necessary. The coarse corpus labels of §3 summarize where failures occur, but they do not specify a check: “gradient synchronization”, for example, does not say which states should agree, across which ranks, or at what point in the step. Writing one check per incident would preserve these accidental details and transfer poorly. At the other extreme, asking the LLM to invent a relation while inspecting each framework gives the miner an unbounded and moving search space. The Pattern Catalog bounds

²Terminology conventions. Schema collection tiers S0–S6 denote trace field sets and their cost trade-offs; rule-trigger tiers A0/A1 denote the firing layer (A0 = PyTorch public interface, A1 = framework adapter).

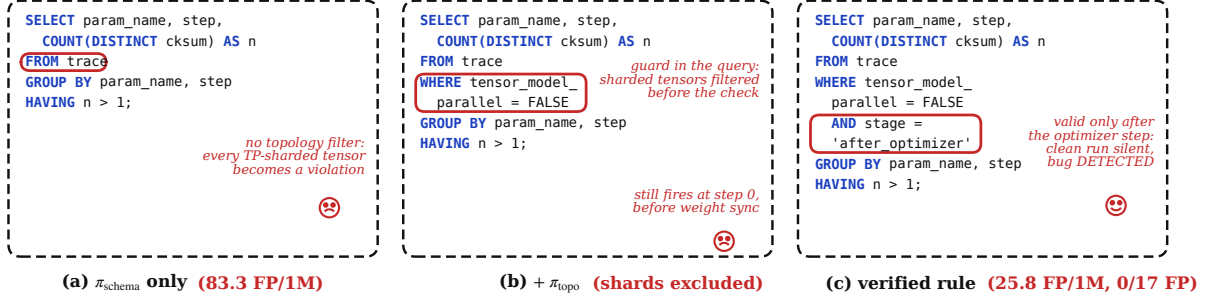


Figure 4: Topology- and phase-guarded SQL compilation for cross-rank replication.

this search by maintaining the relation families currently supported by the evidence, while leaving framework objects and applicability conditions to later grounding.

Catalog construction and use. Each catalog entry records a framework-independent relation (e.g., equality or ordering), its semantic object roles and observable violation witness, and the topology and phase guards under which it may apply; we admit entries supported by recurring bug-and-fix evidence and version the catalog as new evidence arrives. The Invariant Miner then grounds a selected entry into framework-specific fields, topology groups, and lifecycle stages and verifies the resulting executable rule before deployment; we evaluate a frozen catalog snapshot in §5.4, with the complete construction protocol in Appendix D.

Template representation and boundary. A catalog entry separates invariant semantics from framework-specific binding. For example, the cross-rank replication template records equality as its relation, a replicated parameter as its semantic object, multiple checksums within one replica group as the violation witness, and open slots for the topology group, sharding predicate, and lifecycle point. These fields map directly to the three-predicate interface: the relation and witness define the schema-level violation, the comparison group supplies π_{topo} , and the sharding and lifecycle slots supply π_{precond} . Framework parameter names, attributes, and hookpoints remain open until grounding binds them from target-source evidence. Two incidents share a template only when their relation, semantic roles, and violation witness agree; differences in topology or phase become guard slots. We split candidates whose merger changes the relation, requires unrelated disjunctions, lacks a common witness, or would flag a legal execution. A recurring catalog entry requires support from two independent incidents; one-off relations remain singleton evidence.

4.3 Invariant Miner: Instantiating Templates into Verified Rules

A catalog template names a relation worth checking, but an executable rule must scope that relation to the code, topology, and training phase of a concrete framework. Direct LLM-generated scopes are unsafe: a plausible candidate may omit a topology guard, apply at the wrong phase, or encode a documentation-level assumption that the real initialization path violates. The Invariant

Miner therefore treats every LLM-instantiated candidate as *untrusted until verified* and admits it into the verified constraint library $\mathcal{P}$ only after counterexample verification, healthy-run replay, and topology-aware scope validation. Candidate generation is bounded to instantiating $\pi_{\text{schema}} \wedge \pi_{\text{topo}} \wedge \pi_{\text{precond}}$ from a catalog template.

Candidate verification proceeds through four ordered stages. The miner first **scopes** the candidate to a training phase and parameter class, fixing $\pi_{\text{topo}} \wedge \pi_{\text{precond}}$; without this step, a cross-rank equality rule would also match correctly sharded parameters. Only once the scope is constrained does it **ground** π_{schema} , collecting supporting evidence $\mathcal{E}^+$ and contradicting evidence $\mathcal{E}^-$ from the call sites and module attributes retrieved from the framework source. The grounded candidate then enters the adversarial **construct** stage, which synthesizes at least two counterexamples $\mathcal{CE}$ that would make the rule fire on a clean run if its scope were still too broad. Finally, the miner **accepts** the candidate only when no counterexample validates and the confidence score clears a threshold:

$$\begin{aligned} \text{ACCEPT}(c) \Leftrightarrow & (\forall ce \in \mathcal{CE} : \text{VERIFY}(ce) \neq \text{HOLDS}) \\ & \wedge (\text{CONF}(c) \geq \theta_{\text{conf}}), \end{aligned} \quad (3)$$

A single confirmed counterexample is *decisive*: a constructed state on which the candidate fires during a provably correct execution refutes semantic invariance (Definition 3.2) and rejects the candidate regardless of its confidence score. $\text{CONF}(c) \in [0, 1]$ plays the complementary *screening* role, aggregating evidence strength and counterexample-test outcomes on top of a hard floor of two independent evidence sources. The score is uncalibrated and serves only to keep thin-evidence survivors out of the library, with $\theta_{\text{conf}}=0.8$ as a conservative default (full rule in Appendix E.2). Precision is evaluated separately end to end: the adversarially verified cohort fires on 0 of 764 held-out clean evaluations, and the deployed library produces 0/17 (0.0%) fixed-side false positives (§5.3).

The mining loop is a five-stage finite-state machine covering gap analysis, evidence retrieval, synthesis, persistence, and reporting (state diagram in Appendix E); S_3 is the only branching stage and hosts the verification funnel above. On a catalog or framework update, unchanged templates and adapters are reused while affected scopes are re-grounded and verified (cost in §5.5). The miner and runtime pruner divide responsibility for scope enforcement: the funnel writes the tightest scope supported by static evidence, and

Algorithm 1 Counterexample-guided constraint mining loop.

Require: Pattern catalog C , framework source F , launch script L
Ensure: Verified constraint library $\mathcal{P}$

```

1:  $\mathcal{P} \leftarrow \emptyset$  // initialize library
2: while coverage_gap( $C, \mathcal{P}$ ) > 0 do
3:    $pat \leftarrow S_1.PICKPATTERN(C, \mathcal{P})$  //  $S_1$ : gap analysis
4:    $ctx \leftarrow S_2.GATHEREVIDENCE(pat, F, L)$  //  $S_2$ : evidence gathering
5:    $c \leftarrow S_3.SYNTHESIZE(pat, ctx)$  //  $S_3$ : candidate proposal
6:    $CE \leftarrow S_3.ATTACK(c)$  //  $S_3$ : counterexample attack
7:   if ( $\forall ce \in CE : \text{VERIFY}(ce) \neq \text{HOLDS} \wedge (\text{CONF}(c) \geq \theta_{\text{conf}})$ ) then
8:      $\mathcal{P} \leftarrow \mathcal{P} \cup \{c\}$  //  $S_4$ : persist accepted
9:   end if
10:   $S_5.REPORT(c, \mathcal{P})$  //  $S_5$ : feedback (accept/reject)
11: end while
12: return  $\mathcal{P}$ 

```

at job start the pruner reads the live τ and disables rules whose preconditions do not match the live configuration.

For cross-rank replication, the miner leaves the rule with $\pi_{\text{topo}} = [\text{TP}>1]$ and $\pi_{\text{precond}} = [\text{tpl}=\text{False}]$, where tpl marks intentional tensor-parallel sharding of the parameter.

Figure 4 shows how the running replication check changes as its guards are compiled into SQL. Panel (a) uses only π_{schema} and reports legitimately TP-sharded tensors; panel (b) adds π_{topo} but can still fire before step-1 synchronization; panel (c) adds the phase precondition, leaving the SwitchMLP router .weight violation as a non-empty query result. The matched ablation in §5.3 quantifies this progression (83.3 to 25.8 FP/1M and 0/17 fixed-side false positives for the guarded rule).

4.4 Training Instrumentation: Capturing the Runtime Fields Rules Need

Once a rule is accepted, its required runtime fields determine collection cost. The Data Collector therefore lets the verified constraint library $\mathcal{P}$ select those fields instead of imposing one fixed trace schema: rules requiring only checksums and rank coordinates keep the collector at tier S0, while rules needing optimizer state or control variables enable later tiers. Table 3 organizes S0–S6 as ordered schema tiers; the cost of raising a tier is evaluated in §5.5 and the full coverage–overhead curve is given in Appendix I (Figure I.1).

The collector instruments the PyTorch training loop at five canonical lifecycle points—before-forward, after-forward, main-grad-in-backward, after-backward, and before-optimizer—without modifying user code; three auxiliary taps (checkpoint save/load, distributed.all_reduce, build snapshot) extend coverage to the 8 hookpoints used by the verifier’s indexer (§5.5), which include both optimizer pre- and post-step taps. The dominant overhead source is not the number of hooks but *what* each hook computes: full-tensor checksums require moving data to the CPU, so the collector uses GPU-side scalar reductions whenever a rule does not require bit-exact checksums. Captured records are written, with rank/step/stage coordinates unrolled into columns, into a DuckDB-backed trace database with a two-step sliding window—the faults we target reproduce deterministically under a fixed configuration and seed, so cross-rank and cross-step checks need at most two consecutive snapshots. For the running example, the cross-rank replication rule requires only the S0 columns (cksum, param_name, rank, step, stage) at the after-optimizer (optimizer post-step) tap.

Table 3: Opt-in trace-schema tiers exposed by the Data Collector.

Schema tier	Representative new fields
S0	cksum, param_name, ranks, step, stage, dtype, shape
S1	grad_norm, grad_cksum
S2	loss_value
S3	learning_rate, micro_step_id
S4	optimizer_state_cksum
S5–S6	ep/cp_rank, zero_stage, has_nan/inf, num_tokens

4.5 Online Detection: Compiling Verified Rules into Runtime Checks

After the required fields are written to the trace, the Verifier evaluates the accepted rules. Rather than evaluating each rule as a separate Python predicate with separate paths for state snapshots, call events, and initialization checks, TRAINAUDIT compiles instantiated rules into parameterized SQL over the DuckDB trace: π_{topo} and π_{precond} become WHERE filters, and π_{schema} becomes the violation condition in the HAVING clause. An empty result means the constraint holds for the current step; a non-empty result is the violation set, already labeled with the offending parameter, step, and ranks.

Before the first training step, the verifier performs three tasks. First, it extracts the active topology τ from the launch script—DP/TP/PP/EP group sizes, zero_stage, and mixed-precision dtype—and disables rules whose π_{topo} cannot hold under it. Second, it builds a topology-aware lineage table that marks each parameter as replicated or intentionally sharded, so SQL filters remove sharded weights before π_{schema} is evaluated; this is the runtime counterpart of the SwitchMLP scope in §4.1. Third, it compiles every constraint into a parameterized SQL query. State-row, call-event, and initialization-snapshot rules differ only in which trace relations they read.

After each eligible event or scheduled snapshot, the Data Collector commits its records and the verifier executes the surviving rules bound to that hookpoint; event-triggered rules therefore run every step, whereas snapshot-bound rules follow the configured sampling period analyzed in §5.5. State-equality, call-ordering, and initialization violations all compile to the same query shape (full SQL specification in Appendix F).

When a query returns a non-empty row set, the verifier walks the corresponding rule’s coarse-to-fine dependency tree to localize the fault: coarse rules (whole-tensor checksums) run first, and finer rules (per-module, per-stage, per-parameter, bitwise) instantiate only after a coarse check has confirmed that a fault is likely. The final report names the violating constraint, the affected parameter, the divergent ranks, and the step at which the violation first appeared. On the detected Real-SE cases, rule names alone localize 53% of cases, and the expand–contract drill-down compresses the remaining candidate sets by $\sim 6\times$. Appendix G gives the full quantitative study.

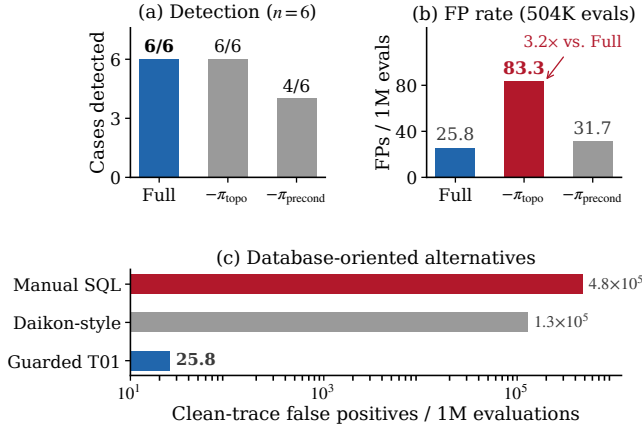


Figure 5: Guard conditions and database-oriented baselines. Panels (a–b) report the matched predicate ablation; panel (c) compares clean-trace FP rates over the same trace relations on a log scale.

5 EVALUATION

We evaluate whether (1) verified guarded constraints detect real silent errors that prior detectors miss, (2) each guard and validation stage controls candidate quality and deployment size, (3) the Pattern Catalog can be constructed reproducibly and covers held-out records, (4) the checks run at practical cost, and (5) the resulting library transfers across frameworks and versions. Real-SE replays run on one eight-NVIDIA-H800 node; end-to-end overhead runs span 4–32 nodes with eight H800 GPUs each (32–256 GPUs), while full-snapshot microbenchmarks use a separate NVIDIA-H20 server with a 1.2B-parameter GPT harness.

5.1 Experimental Setup

Our protocol separates catalog construction from detector scoring and makes execution-protocol differences explicit.

Benchmark and baselines. Real-SE is a frozen, coverage-constrained detector benchmark for held-out recall within the 392-record catalog corpus. It contains 17 detector-coverable paired reproductions of real buggy and fixed revisions and 1 predesignated observability-boundary case. Before catalog induction, every Real-SE incident was assigned to the locked test in the 128/186/78 seed/development/locked split. The benchmark covers held-out catalog families; prevalence and out-of-pool generalization are outside its scope. Construction, admission, and replay details appear in Appendix B.5. We compare against Naïve Monitoring (loss, gradient norm, and NaN/Inf), the official TrainCheck [13] implementation, Manual SQL equality, bound, and monotonicity checks, and leave-one-configuration-out Daikon-style mining [7] without topology guards or counterexample verification.

Comparison and verdict protocol. TRAINAUDIT and the database-style baselines consume traces from these paired reproductions. TrainCheck instead infers case-local invariants from a fixed full-training surrogate and checks its aligned buggy surrogate; Naïve Monitoring consumes the same surrogate metrics. These surrogate

runs provide an *oracle-coverage study*; Appendix I gives the per-case mappings.

A TRAINAUDIT detection must be the first alert on the buggy replay and identify the ground-truth rule and parameter; any alert on the paired fixed side is a false positive. Requiring the ground-truth firing to appear first models an operator triaging alerts in order: a stream led by spurious findings does not provide a usable diagnosis even if the real violation appears later. This first-alert convention affects only ablations, because the deployed library raises no spurious Real-SE alert. We freeze the library without case identifiers, fixes, or verdicts and report buggy-case detection, fixed-side FP per case, and clean-trace FP per million rule evaluations. TrainCheck’s fixed traces serve as inference inputs, whereas clean-trace FP uses independently collected healthy runs. Detection and clean-trace precision therefore rest on different evidence.

The paired reproductions run on the H800 node with NVLink, PyTorch v2.1, and NCCL v2.18 unless an upstream revision requires otherwise; each case executes at its own buggy/fixed revisions rather than under one global framework version.

5.2 Detection Effectiveness and Precision

Real-bug detection. As Table 4 shows, TRAINAUDIT detects 17 of 18 cases: all 17/17 detector-coverable commit-pair cases and **94.4%** overall (95% Wilson CI [74.2%, 99.0%]), counting O-003 as a miss (per-case outcomes in Appendix I). Because several cases share a framework or rule, the interval’s independence assumption is optimistic; requiring executable commit pairs and compact drivers also over-represents bugs with localizable method-level witnesses.

Table 4: Real-SE evidence under two execution protocols. Panel (a) evaluates TRAINAUDIT on paired framework revisions; panel (b) reports TrainCheck and Naïve Monitoring oracle coverage on case-aligned surrogates (17 completed cases; O-003 pending).

(a) Paired buggy/fixed revisions.		
Method	Buggy cases detected	Fixed-side FP
TRAINAUDIT (this work)	17/18 (94.4%)	0/17 (0.0%)
(b) Case-aligned surrogate oracle coverage.		
Method	Cases covered	
TrainCheck [13]	5/17 (29.4%)	
Naïve Monitoring	0/17 (0.0%)	

Under the case-aligned surrogate protocol, TrainCheck detects **5/17 (29.4%)** cases (11 misses and the O-005 inference failure), while Naïve Monitoring detects **0/17**. None of the evaluated faults moves loss, gradient norm, or NaN/Inf counts beyond the thresholds used by Naïve Monitoring. TrainCheck’s measured coverage reflects the API traces and relation templates exposed by the selected surrogate runs. In each miss, the collected trace or inferred templates omit the fault-relevant relation; broader example runs may expose additional relations.

Database-oriented baselines. Figure 5(c) asks whether standard constraints over the same trace relations suffice. Manual SQL expresses self-contained equality, bound, or monotonicity checks,

Table 5: Database-oriented alternatives over the same trace relations. The 25.8 FP/1M entry is the guarded T01 residual; the six-rule deployment mix is silent on all 777,220 evaluations.

	Manual SQL	Daikon-style	TRAINAUDIT (verified)
Topology guard	no	no	yes
Counterexample verification	no	no	yes
Clean-trace FP/1M	4.8×10^5	1.3×10^5	25.8

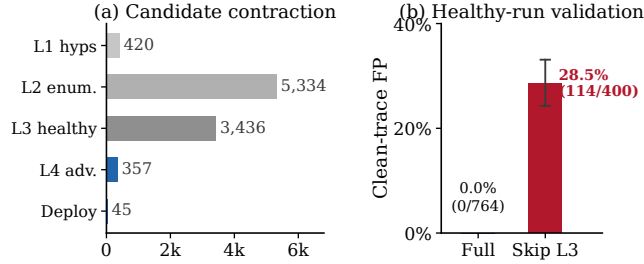


Figure 6: Verification funnel: candidate contraction through verification and curation (a), and held-out clean firing with and without healthy-run validation (b).

while generic Daikon-style templates add learned single-trace relations; neither supplies the topology and phase guards needed to deploy the relations safely. On four clean Megatron-LM configurations, unguarded equality misclassifies 48% of multi-replica groups (22,287/45,956; 4.8×10^5 FP/1M) because many tensors are intentionally sharded. Daikon-style templates, learned leave-one-configuration-out, retain 1.3×10^5 FP/1M, dominated by workload-specific ranges and equality over sharded tensors. Guarded T01-style checks reduce that residual to 25.8 FP/1M. A six-rule deploy mix on the same four stores is silent on 777,220 rule evaluations (0/777,220; 95% Wilson upper bound ≤ 2.5 per million). That denominator counts evaluations ($4 \times 194,305$), not store rows ($4 \times 370,270$).

Observability boundary. O-003 exhibits only sub-percent z-loss drift and changes no field that the online schema can stably distinguish without a trusted reference; it is therefore an observability limit rather than a missing rule. On the precision side, TRAINAUDIT produces no false alert in all 17 paired fixed-side evaluations.³

5.3 Impact of Guarded Verification

We isolate two decisions: whether runtime rules need both guards, and whether candidate constraints need healthy-run and adversarial verification. A complementary leave-one-database-out ablation over 42 databases preserves the same ordering in every database: removing π_{precond} , adversarial verification, and π_{topo} raises aggregate false positives from 342 to 429, 551, and 598, respectively (funnel detail in Appendix E).

Predicate-level effect. We evaluate the three configurations on the six-case Real-SE multi-rank subset and 504 K clean rule evaluations

³Sixteen fixed sides complete cleanly; M-020’s upstream fix instead rejects the faulty configuration as intended. We count this as a fixed-side verdict, not as a completed CLEAN replay.

Table 6: Direct Catalog ablation with downstream validation held fixed.

Arm	Validated yield	Deploy	Real-SE det.	Fixed FP
Catalog	6.69%	45	17/18	0/17
Free-form	2.00%	29	11/18	2/17

(Figure 5). Topology awareness is critical to precision: removing π_{topo} increases clean-trace false positives by $3.2\times$ (25.8 to 83.3 FP/1M), while preserving 6/6 fault detection. Without π_{precond} , premature step-0 equality alerts precede the real violation in two cases, which the first-alert convention of §5.1 scores as misses (4/6). Consistent with Proposition 3.4, the underlying violation set still grows when the guard is dropped; what the guard removes is the spurious mass that arrives ahead of the true firing. The guarded configuration is the only one with 6/6 detection and fewer than 30 FP/1M.

Verification funnel. Across four frameworks, the nested automated cohorts contract from 5,334 enumerated candidates to 3,436 healthy-run survivors and then 357 adversarially verified candidates (Figure 6). Because each cohort contains the next, buggy-case detection remains **17/18 (94.4%)** throughout; the residual miss is the observability-boundary case of §5.2. What changes is candidate quality. Skipping healthy-run validation yields 114/400 firings on held-out clean traces, whereas the L3 and L4 survivors produce 0/6,922 and 0/764 alerts. Thus L3 removes clean-trace false positives, while adversarial filtering removes 3,079 additional candidates that contribute no Real-SE detection. The final 45-entry deployment library is a separately curated cross-framework subset of L4, not a fifth automated gate, and preserves **0/17 (0.0%)** fixed-side false positives.

5.4 Pattern-Catalog Reproducibility and Generalization

Matched mining ablation. Under the matched evaluation protocol, the Catalog raises validated-rule yield from **2.00%** to **6.69%** and Real-SE detection from **11/18** to **17/18**. All six discordant cases are additional detections by the Catalog, with none favoring the free-form arm (two-sided exact McNemar $p=0.031$). The complete funnel and paired outcomes appear in Appendix D; fixed-side FP is 0/17 for the Catalog arm and 2/17 for free-form mining.

We test generalization in two complementary ways. First, for each of 249 bugs added after the catalog snapshot, we ask whether a frozen template matches the fault’s runtime relation without using that bug during induction. The catalog covers **95.1%** of runtime-observable cases (83.1% overall), and at a matched budget of 15 templates retains a 67.5% versus 47.8% advantage over free-form mining (Figure 7). This advantage reflects curation: 1,841 free-form proposals collapse into coarse buckets, whereas the catalog preserves 35 fine-grained obligations. Second, a locked test on 78 of the original 392 records measures induction reproducibility and groundability: schema match is **92.3%** (72/78), joint grounding is **91.0%** (71/78), and two-rater assignment agreement is $\kappa=0.954$ (Appendix D). Real-SE incidents belong to this locked test, not the 249-bug expansion.

Table 7: Query-path scaling along store-size and loaded-rule axes; reported latencies are per statement, not per tick.

(a) Query path vs. store size.			
Path	12,500 rows	1,481,080 rows	
Point lookup	0.69 ms	0.89 ms	
Two-step window	1.12 ms	1.57 ms	
Full-table scan	2.00 ms	78.00 ms	

(b) Loaded rules.				
	$n=1$	$n=8$	$n=24$	$n=100$
Scan all	8.9 ms	136 ms	605 ms	2,791 ms
Indexed 3	5.9 ms	89 ms	92 ms	92 ms

Table 8: End-to-end overhead across 32–256 GPUs and the verification-cadence trade-off at 256 GPUs; worst-case detection delay equals the interval.

(a) Scale-out online path (100-step interval).					
GPUs (nodes)	Collect ovh.	Full med. ovh.	Full P95 ovh.	Throughput retained	Peak GPU memory
32 (4)	1.9%	6.3%	7.8%	94.1%	102.7%
64 (8)	2.3%	6.9%	8.9%	93.5%	103.1%
128 (16)	2.0%	6.5%	8.2%	93.9%	102.9%
256 (32)	2.4%	7.2%	9.4%	93.3%	103.4%

(b) Verification cadence at 256 GPUs.			
Interval	Median overhead	P95 overhead	Throughput retained
20 steps	9.5%	11.8%	91.3%
100 steps	6.9%	8.7%	93.5%
380 steps	3.7%	5.2%	96.4%

5.5 Efficiency and Detection Latency

Rule-evaluation cost. The Megatron-LM evaluation snapshot activates 24 applicable rules (12 A0 + 9 A1 + 3 probes) bound to 8 hookpoints; non-applicable entries remain offline until grounded for that framework (Appendix I). Hookpoint indexing evaluates about three rules per tick rather than scanning all 24. That index is what keeps the query path flat as the deployment grows: scanning every loaded rule rises from 8.9 ms at one rule to 2,791 ms at 100, while the indexed path stays at 89–92 ms once eight or more rules are loaded. Indexed point lookups likewise remain sub-millisecond (0.69–0.89 ms) as the store grows to 1,481,080 rows. Table 7 reports the full series on both axes. These are per-statement microbenchmarks and exclude collection cost.

Measured end-to-end overhead. Table 8 measures the complete online path, including collection. Across 4–32 H800 nodes (32–256 GPUs), the full collector–store–verifier path adds **6.3–7.2%** median overhead while retaining 93.3–94.1% of baseline throughput. Scaling from 32 to 256 GPUs increases median overhead by 0.9 percentage points, with peak GPU memory remaining within 3.4% of baseline. At 256 GPUs, moving the verification interval from 20 to 380 steps lowers median overhead from 9.5% to 3.7%, while increasing worst-case detection delay from 20 to 380 steps. These measurements cover the incremental online path. Full diagnostic snapshots use a separate collection mode.

Full diagnostic snapshots. On a separate 1.2B/H20 full-snapshot microbenchmark (DP=2, bf16), moving tensor fingerprinting and summary statistics to the GPU reduces snapshot-enabled iteration

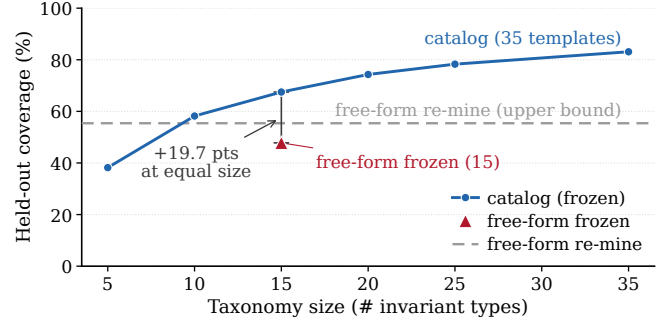


Figure 7: Frozen-Catalog generalization on 249 later-added bugs. At 35 templates the catalog covers 83.1%; at a matched budget of 15 it covers 67.5% versus 47.8% for free-form mining and exceeds the 55.4% re-mining upper bound.

time from 191.9 s to 25.0 s (7.7 $\times$; Appendix I.4). Online deployment amortizes this path through periodic collection; Table 8 reports the end-to-end online cost. The same mechanism shapes the schema tiers of §4.4: because the cost is dominated by the synchronous copy rather than by the number of fields, raising the tier from checksums to GPU-side gradient statistics lowers per-step overhead from ~8% to ~1.5% while widening the observable surface (Appendix I, Figure I.1).

The optimized and SHA-256 paths produce identical cross-rank equality verdicts; only their implementation differs. Event-triggered rules fire on the first recorded event, whereas snapshot-bound rules have a worst-case delay of the dump period K ; because the optimized dump cost ranges from 15 to 51 s across the measured TP/P-P/DP variants, K must be set per topology rather than inferred from world size. Appendix I.4 derives the resulting amortized-overhead curve, on which the optimized collector meets any overhead target at a ~7.7 $\times$ shorter period than the naive one.

Offline cost. Mining Megatron-LM’s deployed rules takes roughly 350 LLM iterations and 40–50 M tokens (about \$70–100 at the evaluated model price); this cost is paid offline and only when the framework library is regenerated.

5.6 Portability and Production Experience

Adapter portability. We deploy grounded rules from the same catalog version through adapters for Megatron-LM, DeepSpeed, OLMo, OLMo-core, and FSDP (Figure 8). The provenance matrix distinguishes direct rule reuse from template reuse with target-specific re-grounding. New rule mining decreases from nine rules for the first framework to none for FSDP, which directly inherits all six framework-agnostic A0 rules through a 30-line adapter. Two A1 rules cannot migrate unchanged because PP partition granularity and replica grouping alter π_{topo} ; their catalog templates remain applicable, but their target-framework topology scopes must be re-grounded and verified.

Transfer analysis. Treating Megatron-LM and DeepSpeed as sources, we apply their rules to a separate transfer set of 10 OLMo/OLMo-core cases assembled for this analysis rather than to the Real-SE replays of §5.2. The source rules detect all eight cases

REFERENCES

- [1] 2026. TrainAudit: full version. https://github.com/vivia-an/trainaudit-vldb-artifact/blob/vldb-2027-artifact-v1.0.1/paper/appendix_supplement.pdf.
- [2] Arvind Arasu, Shivnath Babu, and Jennifer Widom. 2006. The CQL Continuous Query Language: Semantic Foundations and Query Execution. *The VLDB Journal* 15, 2 (2006), 121–142. <https://doi.org/10.1007/s00778-004-0147-z>
- [3] Qixu Chen, Yeye He, Raymond Chi-Wing Wong, Weiwei Cui, Song Ge, Haidong Zhang, Dongmei Zhang, and Surajit Chaudhuri. 2025. Auto-Test: Learning Semantic-Domain Constraints for Unsupervised Error Detection in Tables. *Proc. ACM Manag. Data* 3, 3, Article 133 (2025), 27 pages. <https://doi.org/10.1145/3725396>
- [4] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering Denial Constraints. *Proceedings of the VLDB Endowment (PVLDB)* 6, 13 (2013), 1498–1509. <https://doi.org/10.14778/2536258.2536262>
- [5] Michele Dallachiesa, Amr Ebad, Ahmed Eldawy, Ahmed K. Elmagarmid, Ihab F. Ilyas, Mourad Ouzzani, and Nan Tang. 2013. NADEEF: A Commodity Data Cleaning System. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, USA, 541–552. <https://doi.org/10.1145/2463676.2465327>
- [6] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. 2021. Silent Data Corruptions at Scale. [arXiv:2102.11245 \[cs.AR\]](https://arxiv.org/abs/2102.11245) <https://arxiv.org/abs/2102.11245>
- [7] Michael D Ernst, Jeff H Perkins, Philip J Guo, Stephen McCamant, Carlos Pacheco, Matthew S Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *Science of computer programming* 69, 1-3 (2007), 35–45. <https://doi.org/10.1016/j.scico.2007.01.015>
- [8] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional Functional Dependencies for Capturing Data Inconsistencies. *ACM Transactions on Database Systems (TODS)* 33, 2 (2008), 6:1–6:48. <https://doi.org/10.1145/1366102.1366103>
- [9] Stewart Grant, Hendrik Cech, and Ivan Beschastnikh. 2018. Inferring and asserting distributed system invariants. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg, Sweden, 1149–1159. <https://doi.org/10.1145/3180155.3180199>
- [10] Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. 2021. Cores That Don't Count. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*. ACM, Ann Arbor, MI, USA, 9–16. <https://doi.org/10.1145/3458336.3465297>
- [11] Shuo Hong, Hailong Sun, Xiang Gao, and Shin Hwei Tan. 2024. Investigating and Detecting Silent Bugs in PyTorch Programs. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE. <https://doi.org/10.1109/SANER60148.2024.00035>
- [12] Haitian Jiang, Shaowei Zhu, Zhen Zhang, Zhenyu Song, Xinwei Fu, Zhen Jia, Yida Wang, and Jinyang Li. 2025. TTrace: Lightweight Error Checking and Diagnosis for Distributed Training. [arXiv:2506.09280 \[cs.DC\]](https://arxiv.org/abs/2506.09280) <https://arxiv.org/abs/2506.09280>
- [13] Yuxuan Jiang, Ziming Zhou, Boyu Xu, Beijie Liu, Runhui Xu, and Peng Huang. 2025. Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association, Boston, MA, USA, 313–329. <https://www.usenix.org/conference/osdi25/presentation/jiang>
- [14] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*.
- [15] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proc. VLDB Endow.* 17, 8 (2024), 1939–1952. <https://doi.org/10.14778/3659437.3659449>
- [16] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. PyTorch distributed: experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3005–3018. <https://doi.org/10.14778/3415478.3415530>
- [17] Chang Lou, Dimas Shidqi Parikesit, Yujin Huang, Zhewen Yang, Senapati Diwangkara, Yuzhuo Jing, Achmad Imam Kistijantoro, Ding Yuan, Suman Nath, and Peng Huang. 2025. Deriving Semantic Checkers from Tests to Detect Silent Failures in Production Distributed Systems. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association.
- [18] Yunchi Lu, Youshan Miao, Cheng Tan, Peng Huang, Yi Zhu, Xian Zhang, and Fan Yang. 2025. TrainVerify: Equivalence-Based Verification for Distributed LLM Training. [arXiv:2506.15961 \[cs.LG\]](https://arxiv.org/abs/2506.15961)
- [19] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, Amsterdam, The Netherlands, 865–882. <https://doi.org/10.1145/3299869.3324956>
- [20] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2018. Mixed Precision Training. In *International Conference on Learning Representations*. OpenReview.net, Vancouver, BC, Canada, 10. <https://openreview.net/forum?id=r1gs9JgRZ>
- [21] Thorsten Papenbrock, Jens Ehrlich, Jannik Marten, Tommy Neubert, Jan-Peer Rudolph, Martin Schönberg, Jakob Zwiener, and Felix Naumann. 2015. Functional Dependency Discovery: An Experimental Evaluation of Seven Algorithms. *Proc. VLDB Endow.* 8, 10 (2015), 1082–1093. <https://doi.org/10.14778/2794367.2794377>
- [22] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 28. PMLR, Atlanta, GA, USA, 1310–1318.
- [23] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 202. PMLR, Honolulu, HI, USA, 27496–27520. <https://proceedings.mlr.press/v202/pei23a.html>
- [24] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2019. Discovery of Approximate (and Exact) Denial Constraints. *Proc. VLDB Endow.* 13, 3 (2019), 266–278. <https://doi.org/10.14778/3368289.3368293>
- [25] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, Atlanta, GA, USA, 1–16. <https://doi.org/10.1109/SC41405.2020.00024>
- [26] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. ACM, Virtual Event, 3505–3506. <https://doi.org/10.1145/3394486.3406703>
- [27] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proc. VLDB Endow.* 10, 11 (2017), 1190–1201. <https://doi.org/10.14778/3137628.3137631>
- [28] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. 2018. Automating Large-Scale Data Quality Verification. *Proceedings of the VLDB Endowment (PVLDB)* 11, 12 (2018), 1781–1794. <https://doi.org/10.14778/3229863.3229867>
- [29] Tariq Siddiqui, Arnd Christian König, Jiashen Cao, Cong Yan, and Shuvendu K. Lahiri. 2025. QURE: AI-Assisted and Automatically Verified UDF Inlining. *Proc. ACM Manag. Data* 3, 1, Article 66 (feb 2025), 26 pages. <https://doi.org/10.1145/3709716>
- [30] Jie Song and Yeye He. 2021. Auto-Validate: Unsupervised Data Validation Using Data-Domain Patterns Inferred from Data Lakes. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, Xi'an, China, 1678–1691. <https://doi.org/10.1145/3448016.3457250>
- [31] Florian Tambon, Amin Nikanjam, Le An, Foutse Khomh, and Giuliano Antoniol. 2023. Silent bugs in deep learning frameworks: an empirical study of Keras and TensorFlow. *Empirical Softw. Engg.* 29, 1 (Nov. 2023), 34. <https://doi.org/10.1007/s10664-023-10389-6>
- [32] Naigang Wang, Jungwook Choi, Daniel Brand, Chia-Yu Chen, and Kailash Gopalakrishnan. 2018. Training deep neural networks with 8-bit floating point numbers. *Advances in neural information processing systems* 31 (2018), 7675–7684. <https://papers.nips.cc/paper/7997-training-deep-neural-networks-with-8-bit-floating-point-numbers>
- [33] Zhengmao Ye. 2026. Fix Incorrect Optimizer Routing for Tied Word Embeddings on the MTP-Only Last PP Stage with Muon. NVIDIA Megatron-LM pull request #4642. Accessed: 2026-08-30.
- [34] Dong Young Yoon, Ning Niu, and Barzan Mozafari. 2016. DBSherlock: A Performance Diagnostic Tool for Transactional Databases. In *Proceedings of the 2016 International Conference on Management of Data*. ACM, San Francisco, CA, USA, 1599–1614. <https://doi.org/10.1145/2882903.2915218>
- [35] Xiao Yu, Haoxuan Chen, Feifei Niu, Xing Hu, Jacky Wai Keung, and Xin Xia. 2025. Towards Understanding Bugs in Distributed Training and Inference Frameworks for Large Language Models. [arXiv:2506.10426 \[cs.SE\]](https://arxiv.org/abs/2506.10426)
- [36] Xuanhe Zhou, Guoliang Li, Zhaoan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2024. D-Bot: Database Diagnosis System Using Large Language Models. *Proc. VLDB Endow.* 17, 10 (2024), 2514–2527. <https://doi.org/10.14778/3675034.3675043>